Python Programming Fundamentals

Debugging in VS Code

Programming Fundamentals Team

SETU

2026-06-12

Outline

Outline

1. What is a Bug?

A **bug** is any error that causes a program to behave incorrectly or crash.

The word "bug" dates to 1947 when a real moth was found causing a relay failure in a Harvard computer. Grace Hopper's team taped it in the logbook.

1. What is a Bug?

A **bug** is any error that causes a program to behave incorrectly or crash.

The word "bug" dates to 1947 when a real moth was found causing a relay failure in a Harvard computer. Grace Hopper's team taped it in the logbook.

Three types of errors in Python:

Type	When detected	Example
Syntax Error	Before running (parse time)	`if x > 0` missing `:`
Runtime Error	While running	`int("hello")` → ValueError
Logic Error	Wrong result (no crash)	Using `+` instead of `*`

Outline

2. Syntax Errors

Python cannot run the program at all — the code is grammatically wrong.

```
# SyntaxError examples
def greet(name)          # Missing colon
    print("Hello")

for i in range(10)       # Missing colon
    print(i)

x = (1 + 2               # Unclosed parenthesis

if x > 0:
    print("positive")
 print("done")            # IndentationError (wrong indent)
```

2. Syntax Errors

2. Syntax Errors

Python cannot run the program at all — the code is grammatically wrong.

```
# SyntaxError examples
def greet(name)          # Missing colon
    print("Hello")

for i in range(10)       # Missing colon
    print(i)

x = (1 + 2               # Unclosed parenthesis

if x > 0:
    print("positive")
 print("done")            # IndentationError (wrong indent)
```

2. Syntax Errors

VS Code **underlines** syntax errors in red even before you run the code. Hover over the red underline to see the error message.

Outline

3. Runtime Errors

The code is syntactically correct but something goes wrong **while running**.

```
# ZeroDivisionError
result = 10 / 0

# ValueError
age = int("hello")

# IndexError
fruits = ["apple", "banana"]
print(fruits[5])

# NameError
print(undefined_variable)
```

3. Runtime Errors

```
# TypeError
total = "10" + 5    # can't add str and int
```

3. Runtime Errors

The code is syntactically correct but something goes wrong **while running**.

```
# ZeroDivisionError
result = 10 / 0

# ValueError
age = int("hello")

# IndexError
fruits = ["apple", "banana"]
print(fruits[5])

# NameError
print(undefined_variable)
```

3. Runtime Errors

```
# TypeError
total = "10" + 5    # can't add str and int
```

Python prints a **traceback** showing exactly where the error occurred and the call stack.

Always read the **last line** of the traceback first — it tells you the error type and message.

Outline

4. Logic Errors

The hardest to find — the program runs without crashing but gives wrong results.

```
# Logic error — wrong formula
def celsius_to_fahrenheit(c):
    return c * 9 / 5 - 32   # Should be + 32, not - 32

print(celsius_to_fahrenheit(0))    # -32 instead of 32

# Logic error — off-by-one
def count_items(n):
    total = 0
    for i in range(n):        # Should be range(1, n+1)
        total += i
    return total

print(count_items(5))   # 10 instead of 15
```

4. Logic Errors

4. Logic Errors

The hardest to find — the program runs without crashing but gives wrong results.

```
# Logic error — wrong formula
def celsius_to_fahrenheit(c):
    return c * 9 / 5 - 32   # Should be + 32, not - 32

print(celsius_to_fahrenheit(0))    # -32 instead of 32

# Logic error — off-by-one
def count_items(n):
    total = 0
    for i in range(n):        # Should be range(1, n+1)
        total += i
    return total

print(count_items(5))   # 10 instead of 15
```

4. Logic Errors

The debugger is essential for tracking down logic errors.

Outline

5. Using the VS Code Debugger

Setting a breakpoint:

1. Click in the **gutter** (left of the line number) — a red dot appears
2. Press `F5` to start debugging
3. The program pauses at each breakpoint

5. Using the VS Code Debugger

Setting a breakpoint:

1. Click in the **gutter** (left of the line number) — a red dot appears
2. Press `F5` to start debugging
3. The program pauses at each breakpoint

Debug toolbar buttons:

Button / Key	Action
`F5` Continue	Run until next breakpoint
`F10` Step Over	Execute current line, stay at same level
`F11` Step Into	Step into a function call
`Shift+F11` Step Out	Finish current function, return to caller
`F9` Toggle Breakpoint	Set or remove breakpoint on current line

Outline

6. The Debug Panels

When paused at a breakpoint, VS Code shows:

Variables panel — all current variable values (local and global)

Watch panel — monitor specific expressions (e.g., `len(my_list)`, `total / count`)

Call Stack panel — shows which functions are currently active and in what order

Debug Console — type any Python expression to evaluate it in the current context

6. The Debug Panels

6. The Debug Panels

When paused at a breakpoint, VS Code shows:

Variables panel — all current variable values (local and global)

Watch panel — monitor specific expressions (e.g., `len(my_list)`, `total / count`)

Call Stack panel — shows which functions are currently active and in what order

Debug Console — type any Python expression to evaluate it in the current context

```
# Example to debug — broken average calculator
def calculate_average(numbers):
    total = 0
    for num in numbers:
        total = total + num    # Set breakpoint here
    average = total / len(numbers)
    return average
```

6. The Debug Panels

```
data = [10, 20, 30, 40, 50]
result = calculate_average(data)
print(f"Average: {result}")
```

Outline

7. Practical Debugging Walkthrough

Step-by-step debugging a broken function:

```
def find_second_largest(numbers):
    """Return the second largest number."""
    sorted_nums = sorted(numbers)     # Step 1: sort ascending
    return sorted_nums[-2]            # Step 2: return second from end

numbers = [3, 1, 4, 1, 5, 9, 2, 6]
print(find_second_largest(numbers))  # Should be 6
```

7. Practical Debugging Walkthrough

7. Practical Debugging Walkthrough

Step-by-step debugging a broken function:

```
def find_second_largest(numbers):
    """Return the second largest number."""
    sorted_nums = sorted(numbers)     # Step 1: sort ascending
    return sorted_nums[-2]            # Step 2: return second from end

numbers = [3, 1, 4, 1, 5, 9, 2, 6]
print(find_second_largest(numbers))  # Should be 6
```

Debugging strategy:

1. Set a breakpoint inside `find_second_largest`
2. Step through — inspect `sorted_nums` in Variables panel
3. Verify `sorted_nums[-2]` is what you expect
4. Check: does this handle duplicates correctly? What if the list has fewer than 2 items?

7. Practical Debugging Walkthrough

7. Practical Debugging Walkthrough

Step-by-step debugging a broken function:

```
def find_second_largest(numbers):
    """Return the second largest number."""
    sorted_nums = sorted(numbers)     # Step 1: sort ascending
    return sorted_nums[-2]            # Step 2: return second from end

numbers = [3, 1, 4, 1, 5, 9, 2, 6]
print(find_second_largest(numbers))  # Should be 6
```

Debugging strategy:

1. Set a breakpoint inside `find_second_largest`
2. Step through — inspect `sorted_nums` in Variables panel
3. Verify `sorted_nums[-2]` is what you expect
4. Check: does this handle duplicates correctly? What if the list has fewer than 2 items?

7. Practical Debugging Walkthrough

Always test edge cases: empty list, single item, all same values, already sorted.

Outline

8. Print Debugging vs Visual Debugger

Print debugging (quick but messy):

```
def process(data):
    print(f"DEBUG: data = {data}")        # temporary debug prints
    result = []
    for item in data:
        print(f"DEBUG: processing {item}")
        result.append(item * 2)
    print(f"DEBUG: result = {result}")
    return result
```

8. Print Debugging vs Visual Debugger

8. Print Debugging vs Visual Debugger

Print debugging (quick but messy):

```
def process(data):
    print(f"DEBUG: data = {data}")        # temporary debug prints
    result = []
    for item in data:
        print(f"DEBUG: processing {item}")
        result.append(item * 2)
    print(f"DEBUG: result = {result}")
    return result
```

Visual debugger is better because:

- No need to modify your code
- Can inspect **all** variables at once
- Can evaluate expressions interactively
- Call stack shows the full execution path

8. Print Debugging vs Visual Debugger

- No risk of forgetting to remove debug prints before submitting

8. Print Debugging vs Visual Debugger

Print debugging (quick but messy):

```
def process(data):
    print(f"DEBUG: data = {data}")        # temporary debug prints
    result = []
    for item in data:
        print(f"DEBUG: processing {item}")
        result.append(item * 2)
    print(f"DEBUG: result = {result}")
    return result
```

Visual debugger is better because:

- No need to modify your code
- Can inspect **all** variables at once
- Can evaluate expressions interactively
- Call stack shows the full execution path

8. Print Debugging vs Visual Debugger

- No risk of forgetting to remove debug prints before submitting

Rule of thumb: use print debugging for quick 30-second checks; use the visual debugger when the bug is non-obvious.

Thanks for Watching - Any questions?